

1. 기본 리소스, 대체 리소스

- 기본리소스 : 장치 구성과 상관없이 기본적으로 사용되는 리소스
- 대체리소스 : 특정한 장치 구성을 위하여 설계된 리소스

2. 스타일과 테마 비교

- 일반적으로 스타일은 특정 뷰나 레이아웃에 사용되고, 테마는 앱 전체에 영향을 미침
- 테마는 스타일을 기반으로 만들어지며, 앱의 일관된 디자인을 제공하는 데 중요한 역할을 함

3. 권한 요청 - 매니페스트 태그 & 메서드

- 매니페스트 권한 요청 태그
 - `<uses-permission android:name="android.permission.권한_이름" />`
 - `<permission>`
- 매니페스트 권한 요청 메서드
 - **ContextCompat.checkSelfPermission()** 권한이 있는지 **확인**
 - **ActivityCompat.requestPermissions()** 사용자에게 권한을 **요청**
 - **onRequestPermissionsResult()** 사용자 응답에 대한 **콜백 메서드**

4. Fragment를 사용하는 이유

- Activity의 화면을 분할하여 구성하기 위해 사용된다.
- 다양한 화면 크기에 대응하고 재사용성을 높인다.

5. MainThread란?

- 화면 갱신과 사용자 입력을 담당한다.
- **UI 스레드(User Interface Thread)**라고도 부름

6. 작업 Thread를 생성하는 방법 3가지

1. **Thread** 클래스를 상속받아서 Thread를 생성
2. **Runnable** 인터페이스를 구현한 후에 Thread 생성자에 전달
3. **AsyncTask** 클래스를 상속받아
 - 스레드 처리 부분
 - UI 처리 부분 을 구분해서 코딩

7. 용어설명

- **Service** : 사용자 인터페이스(UI) 없이 백그라운드에서 실행되는 컴포넌트
- **Broadcast Receiver** : 안드로이드 장치에서 발생하는 많은 이벤트(방송)들을 받는 컴포넌트
- **Content Provider** : 다른 애플리케이션에 데이터를 공급하는 역할을 하는 컴포넌트

8. 시작서비스 시작하는 메서드

- `startService()`
 - 시작 타입의 서비스(Started Service)를 시작

9. 연결서비스 시작하는 메서드

- `bindService()`
 - 연결 타입의 서비스(Bound Service)를 시작

10. 방송의 종류

- ACTION_TIME_TICK : 1분마다 보내진다
- ACTION_TIMEZONE_CHANGED : 시간대 변경
- ACTION_PACKAGE_ADDED : 패키지 추가
- ACTION_PACKAGE_REMOVED : 패키지 삭제
- ACTION_MEDIA_REMOVED : 외부 저장장치 제거
- ACTION_BATTERY_LOW : 배터리 저충전
- ACTION_POWER_DISCONNECTED : 전원 연결 해제
- sendBroadcast(Intent intent) : 일반적인 방송 전달
- sendOrderedBroadcast(Intent intent, String receiverPermission) : 순서가 있는 방송을 전송
- ACTION_TIEM_CHANGED : 현재 시각 설정
- ACTION_BOOT_COMPLETED : 부트 완료
- ACTION_PACKAGE_CHANGED : 패키지 변경
- ACTION_MEDIA_MOUNTED : 외부 저장장치 마운트 완료
- ACTION_BATTERY_CHANGED : 배터리 상태 변경
- ACTION_POWER_CONNECTED : 전원 연결
- ACTION_SHUTDOWN : 파워 오프

11. SQLiteOpenHelper의 onCreate() 메서드에 기술할 내용

- 테이블 구조(스키마)를 정의하는 CREATE TABLE 문을 작성
- 필요 시 초기 데이터 삽입(INSERT) 구문도 포함 가능

12. SQLiteDatabase 객체와 execSQL / rawQuery / Cursor 역할

- SQLiteDatabase 객체 얻기
 - db = helper.getWritableDatabase(); // 쓰기 가능
 - db = helper.getReadableDatabase(); // 읽기 전용
- execSQL() 의 역할
 - 결과를 반환하지 않는 SQL 문을 실행
 - CREATE, INSERT, UPDATE, DELETE 등 DDL, DML 구문 실행
- rawQuery() 의 역할
 - 결과를 반환하는 SELECT 문을 실행
 - 조회 결과를 Cursor 객체로 반환
- Cursor 클래스의 역할
 - SQL 쿼리를 데이터베이스 서버로 전송하고, 실행된 쿼리의 결과 집합(ResultSet)을 관리한다.
 - 커서(Cursor)를 사용하여 한 번에 한 레코드씩 데이터에 접근 가능

13. RoomDB Annotation

- @Entity
 - 데이터베이스의 테이블 구조를 정의하는 클래스 (필드, 기본키 등)
- @Dao (Data Access Object)
 - 데이터에 접근하는 메서드를 정의하는 인터페이스
- @PrimaryKey(autoGenerate = true)
 - 기본키로 설정하고 데이터의 고유 번호(ID)를 자동으로 1씩 증가시킨다.
- @Insert, @Update, @Delete
 - SQL 문을 직접 쓰지 않아도 Room이 알아서 저장/수정/삭제 처리
- @Query("SELECT...")
 - 복잡한 조회 조건이 필요할 때 직접 SQL 문을 작성
- @Database
 - DB 안에 들어가는 Entity(테이블) 목록 정의

14. 콘텐츠 제공자 - MovieProvider 재정의 메서드

ContentProvider 클래스를 상속받은 MovieProvider 클래스에서 **재정의(Override)해야 하는 메서드**:

- onCreate() : 콘텐츠 제공자가 처음 생성될 때 필요한 초기 작업을 수행한다.
- query(...) : 데이터를 조회(SELECT)하고 결과를 Cursor 객체로 반환한다.
- insert(...) : 새로운 데이터를 삽입(INSERT)하고 추가된 레코드의 Uri를 반환한다.
- update(...) : 기존 데이터를 수정(UPDATE)하고 영향받은 행의 개수를 반환한다.
- delete(...) : 데이터를 삭제(DELETE)하고 삭제된 행의 개수를 반환한다.
- getType(Uri uri) : 주어진 URI의 MIME 타입을 반환한다.